

Programming Assignment 1—LLVM IR

Marc Lin

1 1st-loop-iteration Peeling

In function emitIR_LoopPeeling, I implemented 6 steps to change original while loop CFG to the peeling one. First, I made 2 copies of mBlock (current basic block: preheader) and they would be used in different results of conditional branch. Then removed the content in mBlock to be replaced later.

```
// Step 1: replicate 2 copies of current basic block (mm)
BasicBlock* preheaderCondNo = replicate_basicblock(ctx);
BasicBlock* preheaderCondYes = replicate_basicblock(ctx);

// Step 2: remove content of mblock
RemovePreHeader(ctx);
```

Next, we put while.cond in mBlock, in case to do that, I used IRBuilder to create conditional branch as shown below. Additionally, **there is no need to use BasicBlock::Create since the original mBlock is still there.**

```
// Step 3: create a conditional branch instruction at the end of the mblock
//         to connect the two copies of the preheader
auto value = this->mExpr->emitIR(ctx);
IRBuilder<true, NoFolder> builder(ctx.mBlock);
if (!value->getType()->isIntegerTy(1))
    value = builder.CreateICmpNE(value, ctx.mZero);
builder.CreateCondBr(value, preheaderCondYes, preheaderCondNo); // conditional branch in while.cond
```

BranchInst * CreateBr (BasicBlock *Dest)
Create an unconditional 'br label X' instruction.
BranchInst * CreateCondBr (Value *Cond, BasicBlock *True, BasicBlock *False, MDNode *BranchWeights=nullptr, MDNode *Unpredictable=nullptr)
Create a conditional 'br Cond, TrueDest, FalseDest' instruction.
BranchInst * CreateCondBr (Value *Cond, BasicBlock *True, BasicBlock *False, Instruction *MDSrc)
Create a conditional 'br Cond, TrueDest, FalseDest' instruction.

In step 4 we have to append IR for the 1st body loop iteration, so I created a new block called while.cond. The BB of 1st loop body should have an unconditional branch to while.cond.

```
// Step 4: append IR for the 1st loop body iteration
auto condBlock = BasicBlock::Create(ctx.mGlobal, "while.cond", ctx.mFunc);
ctx.mBlock = preheaderCondYes;
this->mLoopStmt->emitIR(ctx);
IRBuilder<true, NoFolder> builderFirstBody(ctx.mBlock);
builderFirstBody.CreateBr(condBlock); // unconditional branch in while.cond
```

Step 5 I implemented IR lowering for the rest of the loop (Same as section

ASTWhileStmt). And last step is to connect two preheader side into while.end, using unconditional branch.

```
// Step 5: IR lowering for remaining loop body iterations
//          and unconditional branch to the while.end
auto body = BasicBlock::Create(ctx.mGlobal, "while.body", ctx.mFunc);
auto endBlock = BasicBlock::Create(ctx.mGlobal, "while.end", ctx.mFunc);
ctx.mBlock = condBlock;

// set Condition for while.cond
auto valueRemain = this->mExpr->emitIR(ctx);
IRBuilder<true, NoFolder> builderCond(ctx.mBlock);
if (!valueRemain->getType()->isIntegerTy(1))
    valueRemain = builderCond.CreateICmpNE(valueRemain, ctx.mZero);
builderCond.CreateCondBr(valueRemain, body, endBlock);
```

```
// Step 6: unconditional branch to while.end
ctx.mBlock = preheaderCondNo;
IRBuilder<true, NoFolder> builderEnd(ctx.mBlock);
builderEnd.CreateBr(endBlock);

ctx.mBlock = endBlock;
```

2 IR Lifting

This section I basically used ASTPrint.cpp as my reference, and also looked at ASTNodes.h to check each member function usage. In ASTtoCode.cpp the lifting process is similar to grammar in USCLanguage.pdf. As a recursive descent parser, my implementation follows the derivation of grammar, **function ASTtoCode will derive more tokens until it reaches terminal, such as ASTConstantExpr, ASTStringExpr, ASTIdentExpr...etc**, in this case we check member function and use getText(), getName() and put them in final output.

Besides, the semicolon caused some issues. In ASTForStmt, the iteration step (mStepStmt) is an ASTExprStmt instance, but ++i should not end with semicolon. This somehow contradict to usual statement ++i; (ASTIncExpr), Therefore, I use the variable depth to design a control statement, if this ExprStmt is derived from ASTForStmt, it will pass a negative depth, and if the height is negative, we will not put a semicolon at the end.

```

ASTTOCODE(ASTForStmt) // "ForStmt"
// PA1: Implement
OUTS();
output << "for (";
minitStmt->ASTtoCode(output, depth + 1);
mcondExpr->ASTtoCode(output, depth + 1);
output << "; ";
mstepStmt->ASTtoCode(output, -1); // avoid extra semicolon
output << ")" {" << std::endl;
mbodyStmt->ASTtoCode(output, depth + 2);
OUTS();
output << "}" << std::endl;
}

```

```

ASTTOCODE(ASTExprStmt) // "ExprStmt"
// PA1: Implement
if (depth >= 0) {
    mExpr->ASTtoCode(output, depth + 1);
}
else {
    mExpr->ASTtoCode(output, -1);
}
}

```

```

ASTTOCODE(ASTIncExpr) // "IncExpr: "
// PA1: Implement
if (depth >= 0) {
    OUTS();
    output << "++" << mIdent.getName();
    output << ";";
}
else {
    output << "++" << mIdent.getName();
}
}

```

3 InstCombine Pass

For 5 kinds of optimization, I designed the cases as 1 -> 3 -> 2/4 -> 5, part 1 is for both lhs and rhs are ConstantInt, so they don't get other chances to be optimized by other parts. Part 2~4 are all for lhs **or** rhs is constant. Part 5 is only for same operand, and this might happened after case 2~4 optimization, so I put it in the last part.

For controlling the optimization part, I found out that decide whether lhs and rhs are ConstantInt first, then use switch-case to obtain which kind of Instruction is currently being optimized is a better flow. Below is how I implemented the iteration over instructions in functions.

```

// PA1: Implement Part 1 ~ 5 of instcombine optimization
for (BasicBlock& BB : F) { // Iterate basic block in function
    for (Instruction& I : BB) { // Iterate instruction in the basic block
        BinaryOperator *BI = dyn_cast<BinaryOperator>(&I);
        if (BI) { // if not BinaryOpt, return nullptr to BI
            ConstantInt *lhs = dyn_cast<ConstantInt>(BI->getOperand(0)); // if not ConstantInt
            ConstantInt *rhs = dyn_cast<ConstantInt>(BI->getOperand(1)); // if not ConstantInt

            // Part 1: Local Constant folding
            if (lhs && rhs) {

```

For part 3 optimization, worklist-reverse-iteration algorithm is for cases if StoreInst and LoadInst are in different BasicBlocks. But we should search for the same BB first, then we go for the Algorithm 1. Therefore I combined the same BB search in Algorithm 1, without changing the argument (LoadInst) and return value (StoreInst). The rest of the function is implemented as the pseudocode.

```

StoreInst *reAssociate(LoadInst *LI)
{
    Value *loadPtr = LI->getPointerOperand();
    BasicBlock *startBB = LI->getParent(); // getParent(): get

    // (1) first, search in the same basic block
    BasicBlock::iterator it(LI);
    while (it != startBB->begin()) {
        --it;
        if (StoreInst *SI = dyn_cast<StoreInst>(&*it)) {
            if (SI->getPointerOperand() == loadPtr) {
                return SI;
            }
        }
    }
}

```

after we get the closest StoreInst for the current LoadInst, there are still 2 cases: (1) if associate by register, and (2) if associate by memory (stack). The algorithm is for case (2), so we use a control statement as below:

```

else if (rhs) {
    // Part 3: Re-association of Nested Expressions to Fold Constants
    if (BI->getOpcode() == Instruction::Add || BI->getOpcode() == Instruction::Mul) {
        Value* opLhs = BI->getOperand(0);
        BinaryOperator* innerBI = nullptr;

        // (1) if associate by register
        if (BinaryOperator* tempBI = dyn_cast<BinaryOperator>(opLhs)) {
            innerBI = tempBI;
        }

        // (2) if associate by memory (stack)
        else if (LoadInst* LI = dyn_cast<LoadInst>(opLhs)) {
            if (StoreInst* SI = reAssociate(LI)) {
                innerBI = dyn_cast<BinaryOperator>(SI->getValueOperand());
            }
        }
    }
}

```

After deciding inner Binary Instruction (innerBI), do constant folding.

```

// optimize by constant folding
if (innerBI && innerBI->getOpcode() == BI->getOpcode()) {
    if (ConstantInt *C1 = dyn_cast<ConstantInt>(innerBI->getOperand(1))) {
        APInt combinedVal;
        if (BI->getOpcode() == Instruction::Add)
            combinedVal = C1->getValue() + rhs->getValue();
        else
            combinedVal = C1->getValue() * rhs->getValue();

        ConstantInt *C3 = ConstantInt::get(BI->getContext(), combinedVal);
        BinaryOperator *newBI = BinaryOperator::Create(
            BI->getOpcode(),
            innerBI->getOperand(0),
            C3,
            "",
            BI
        );
        BI->replaceAllUsesWith(newBI);
        toErase.push_back(BI);

        changed = true;
        continue;
    }
}

```

Review 1 file >

And this is the part 3 implementation.

After each part of optimization, if the code has changed, we use *didCalc* to represent the instructions should be replaced, using *replaceAllUsesWith* (), then we use *changed* and *toErase* as the final decision of erasing the instructions.

```

if (didCalc) {
    // Replace the instruction with the constant result
    BI->replaceAllUsesWith(ConstantInt::get(BI->getType(), result));
    // Do not erase the instruction when iterating over the basic block
    toErase.push_back(&I);
    changed = true;
}

```

toErase is a vector of Instruction, which record all the instructions that will actually be erased. They are initialized at the beginning.

```

bool changed = false;
std::vector<Instruction*> toErase;

```

After all parts of optimization, the last part of InstCombine::runOnFunction:

```

// Safe deletion after iteration
for (Instruction* inst : toErase) {
    inst->eraseFromParent();
}
return changed;

```